

Name: Manahil Habib

Reg no: 2023-BSE-042

Section: 5B

Cloud Computing

Assignment# 02

Executive Summary

This report covers the implementation of Assignment 2, which focuses on deploying a multi-tier web environment on Amazon Web Services (AWS) using Terraform. The goal is to gain hands-on experience with Infrastructure as Code (IaC), cloud networking, and advanced Nginx configurations.

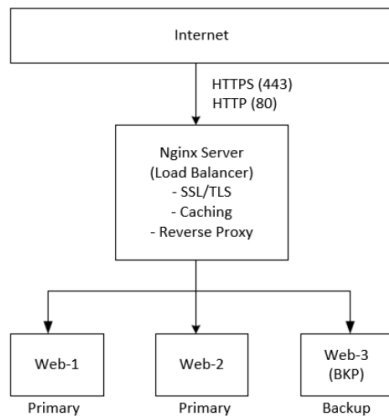
The setup consists of a high-availability web infrastructure on AWS. It includes a single Nginx server functioning as a reverse proxy and load balancer, along with three Apache backend servers. The Nginx server is configured with HTTPS via a self-signed SSL certificate, caching for performance optimization, and load balancing to distribute requests across backend servers. One backend server is designated as a backup to ensure continuity in case primary servers fail.

Terraform modules were utilized to structure the project cleanly, with separate modules for networking, security, and web servers. The networking module sets up the VPC, subnets, internet gateway, and routing. Security groups are defined in the security module to permit only required traffic, following best practices. The webserver module is reused for Nginx and backend servers alike.

Server configuration is automated using shell scripts. Apache backend servers display system and network metadata, while Nginx handles SSL, caching, traffic forwarding, and failover. The deployment is tested for load balancing, caching, backup server functionality, and security compliance.

This assignment reinforced practical skills in Terraform, AWS services, and Nginx configuration, while enhancing understanding of automation, scalability, and high-availability design in cloud systems.

Architecture Overview



Part 1: Infrastructure Setup (25 marks)

1.1 Project Structure (5 marks)

Tasks:

- Create all necessary files and directories
- Implement proper .gitignore to exclude sensitive files
- Document the project structure in README.md

Deliverables:

- Screenshot: assignment_part1_project_structure.png (tree command output)

```
PS C:\Users\HABIB TARIQ\Documents\Assignment2> tree /F
Folder PATH listing
Volume serial number is F256-9870
C:.\
|   locals.tf
|   main.tf
|   outputs.tf
|   README.md
|   terraform.tfvars
|   variables.tf
|
|---modules
|    |---networking
|    |---security
```

- Screenshot: assignment_part1_gitignore.png (content of .gitignore)

```

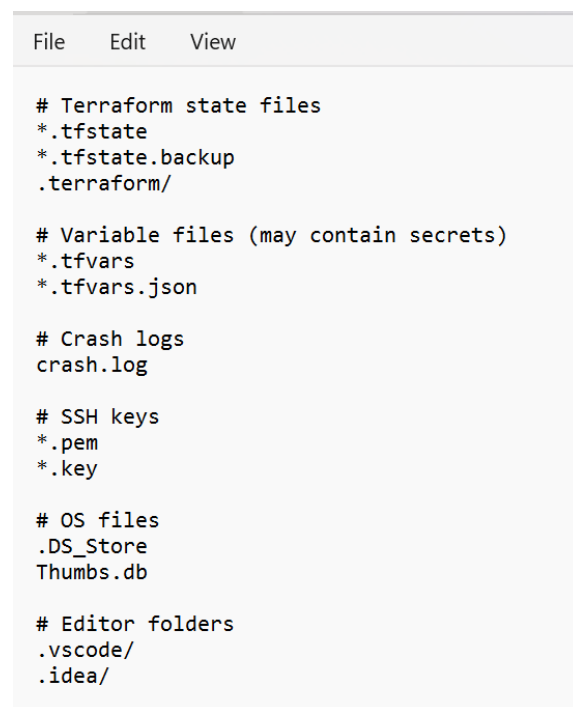
PS C:\Users\HABIB TARIQ\Documents\Assignment2> New-Item -ItemType File -Name ".gitignore"

Directory: C:\Users\HABIB TARIQ\Documents\Assignment2

Mode                LastWriteTime         Length Name
----                -
-a-----         12/30/2025   4:33 PM             0 .gitignore

PS C:\Users\HABIB TARIQ\Documents\Assignment2> notepad .gitignore

```



```

File Edit View

# Terraform state files
*.tfstate
*.tfstate.backup
.terraform/

# Variable files (may contain secrets)
*.tfvars
*.tfvars.json

# Crash logs
crash.log

# SSH keys
*.pem
*.key

# OS files
.DS_Store
Thumbs.db

# Editor folders
.vscode/
.idea/

```

1.2 Variable Configuration (5 marks)

Define all required variables in variables.tf:

Tasks:

- Add validation rules for CIDR blocks
- Add descriptions for all variables
- Set appropriate defaults where applicable
- Populate terraform.tfvars with your values

Deliverables:

- Screenshot: assignment_part1_variables_tf.png

```
File Edit View

variable "vpc_cidr_block" {
  description = "CIDR block for the VPC"
  type        = string

  validation {
    condition = can(regex("^[0-9]{1,3}\\.[0-9]{1,3}/[0-9]+$", var.vpc_cidr_block))
    error_message = "Please provide a valid CIDR block for VPC."
  }
}

variable "subnet_cidr_block" {
  description = "CIDR block for the subnet"
  type        = string

  validation {
    condition = can(regex("^[0-9]{1,3}\\.[0-9]{1,3}/[0-9]+$", var.subnet_cidr_block))
    error_message = "Please provide a valid CIDR block for subnet."
  }
}

variable "availability_zone" {
  description = "Availability zone for resources"
  type        = string
}

variable "env_prefix" {
  description = "Environment prefix for resource naming"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
}

variable "public_key" {
  description = "Path to public SSH key"
  type        = string
}

variable "private_key" {
  description = "Path to private SSH key"
  type        = string
}

variable "backend_servers" {
  description = "List of backend servers"
  type = list(object({
    name      = string
    script_path = string
  }))
}
```

- Screenshot: assignment_part1_terraform_tfvars.png

```
vpc_cidr_block      = "10.0.0.0/16"
subnet_cidr_block   = "10.0.10.0/24"
availability_zone    = "us-east-1a"
env_prefix          = "prod"
instance_type       = "t3.micro"

public_key = "C:\\Users\\HABIB TARIQ\\.ssh\\id_ed25519.pub"
private_key = "C:\\Users\\HABIB TARIQ\\.ssh\\id_ed25519"

backend_servers = [
  {
    name      = "web-1"
    script_path = "./scripts/apache-setup.sh"
  },
  {
    name      = "web-2"
    script_path = "./scripts/apache-setup.sh"
  },
  {
    name      = "web-3"
    script_path = "./scripts/apache-setup.sh"
  }
]
```

1.3 Networking Module (5 marks)

Tasks:

- Create VPC resource
- Create subnet with `map_public_ip_on_launch = true`
- Create and attach Internet Gateway
- Configure routing table
- Add proper tags to all resources using `env_prefix`

Deliverables:

- Screenshot: `assignment_part1_networking_module_main.png`

```
File Edit View

variable "vpc_cidr_block" {
  description = "CIDR block for the VPC"
  type       = string
}

variable "subnet_cidr_block" {
  description = "CIDR block for the public subnet"
  type       = string
}

variable "availability_zone" {
  description = "Availability zone for subnet"
  type       = string
}

variable "env_prefix" {
  description = "Environment prefix for naming"
  type       = string
}

variable "common_tags" {
  description = "Common tags for resources"
  type       = map(string)
}
|
```

- Screenshot: `assignment_part1_networking_module_outputs.png`

```
File Edit View

output "vpc_id" {
  description = "VPC ID"
  value      = aws_vpc.this.id
}

output "subnet_id" {
  description = "Subnet ID"
  value      = aws_subnet.this.id
}

output "igw_id" {
  description = "Internet Gateway ID"
  value      = aws_internet_gateway.this.id
}

output "route_table_id" {
  description = "Route Table ID"
  value      = aws_route_table.this.id
}
```

1.4 Security Module (5 marks)

Create a security module that provisions:

Tasks:

- Create Nginx security group with appropriate rules
- Create backend security group with appropriate rules
- Use security group IDs for backend ingress (not CIDR blocks)
- Add descriptive names and tags

Deliverables:

- Screenshot: assignment_part1_security_module. Png

File Edit View

```
resource "aws_security_group" "nginx_sg" {
  name           = "${var.env_prefix}-nginx-sg"
  description    = "Security group for Nginx reverse proxy"
  vpc_id        = var.vpc_id

  ingress {
    description = "SSH from my IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip]
  }

  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "HTTPS from anywhere"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    description = "Allow all outbound traffic"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(
    var.common_tags,
    {
      Name = "${var.env_prefix}-nginx-sg"
    }
  )
}

resource "aws_security_group" "backend_sg" {
  name           = "${var.env_prefix}-backend-sg"
  description    = "Security group for backend servers"
  vpc_id        = var.vpc_id

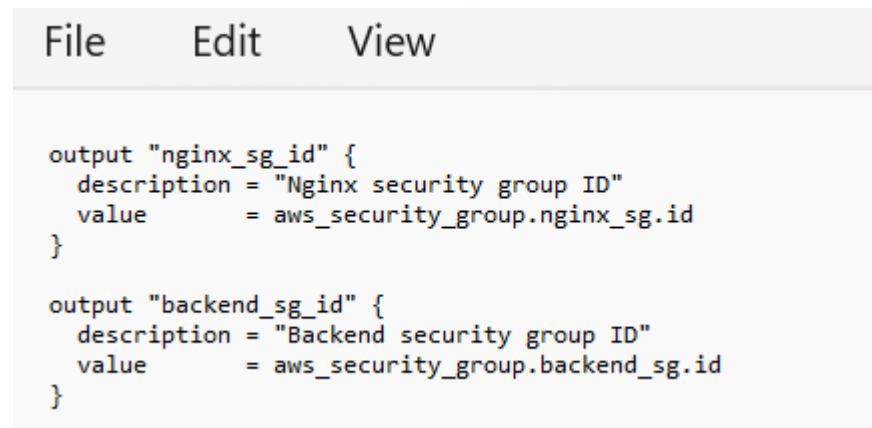
  ingress {
    description = "SSH from my IP"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.my_ip]
  }

  ingress {
    description = "HTTP from Nginx SG"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    security_groups = [aws_security_group.nginx_sg.id]
  }

  egress {
    description = "Allow all outbound traffic"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(
    var.common_tags,
    {
      Name = "${var.env_prefix}-backend-sg"
    }
  )
}
```

- Screenshot: assignment_part1_security_groups_console.png (AWS Console)



```
File      Edit      View

output "nginx_sg_id" {
  description = "Nginx security group ID"
  value       = aws_security_group.nginx_sg.id
}

output "backend_sg_id" {
  description = "Backend security group ID"
  value       = aws_security_group.backend_sg.id
}
```

1.5 Locals Configuration (5 marks)

Tasks:

- Implement dynamic IP detection
- Define common tags
- Define backend server list
- Add any other reusable local values

Deliverables:

- Screenshot: assignment_part1_locals_tf.png



```
#####
# Get My Public IP Automatically
#####
data "http" "my_ip" {
  url = "https://icanhazip.com"
}

#####
# Local Values
#####
locals {

  # My IP with /32 for SSH access
  my_ip = "${chomp(data.http.my_ip.response_body)}/32"

  # Common tags for all resources
  common_tags = {
    Environment = var.env_prefix
    Project     = "Assignment-2"
    ManagedBy   = "Terraform"
  }

  # Backend servers configuration
  backend_servers = [
    {
      name       = "web-1"
      suffix     = "1"
      script_path = "./scripts/apache-setup.sh"
    },
    {
      name       = "web-2"
      suffix     = "2"
      script_path = "./scripts/apache-setup.sh"
    },
    {
      name       = "web-3"
      suffix     = "3"
      script_path = "./scripts/apache-setup.sh"
    }
  ]
}
```

Part 2: Webserver Module (15 marks)

2.1 Module Design (10 marks)

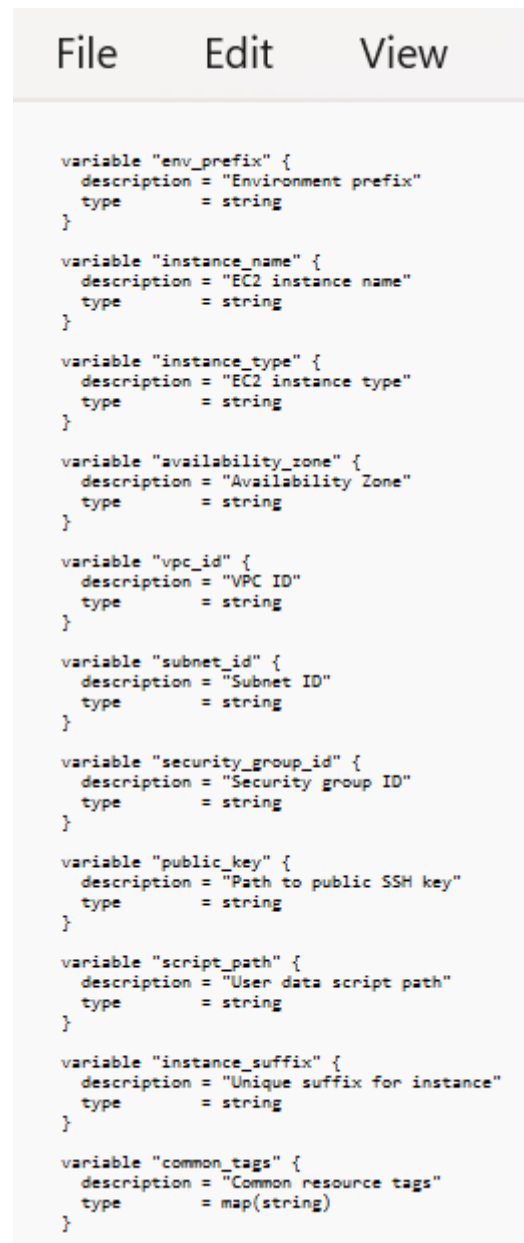
Create a reusable webserver module in modules/webserver/

Tasks:

- Create variables.tf with all required variables
- Create main.tf with key pair and instance resources
- Create outputs.tf with all required outputs
- Ensure module is reusable for both Nginx and backend servers

Deliverables:

- Screenshot: assignment_part2_webserver_module_variables.png



```

File      Edit      View

variable "env_prefix" {
  description = "Environment prefix"
  type        = string
}

variable "instance_name" {
  description = "EC2 instance name"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
}

variable "availability_zone" {
  description = "Availability Zone"
  type        = string
}

variable "vpc_id" {
  description = "VPC ID"
  type        = string
}

variable "subnet_id" {
  description = "Subnet ID"
  type        = string
}

variable "security_group_id" {
  description = "Security group ID"
  type        = string
}

variable "public_key" {
  description = "Path to public SSH key"
  type        = string
}

variable "script_path" {
  description = "User data script path"
  type        = string
}

variable "instance_suffix" {
  description = "Unique suffix for instance"
  type        = string
}

variable "common_tags" {
  description = "Common resource tags"
  type        = map(string)
}

```

- Screenshot: assignment_part2_webserver_module_main.png

File Edit View

```
#####
# Get Latest Amazon Linux 2023 AMI
#####
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

#####
# Key Pair
#####
resource "aws_key_pair" "this" {
  key_name   = "${var.env_prefix}-${var.instance_suffix}-key"
  public_key = file(var.public_key)

  tags = merge(
    var.common_tags,
    {
      Name = "${var.env_prefix}-${var.instance_suffix}-key"
    }
  )
}

#####
# EC2 Instance
#####
resource "aws_instance" "this" {
  ami                  = data.aws_ami.amazon_linux.id
  instance_type        = var.instance_type
  availability_zone     = var.availability_zone
  subnet_id            = var.subnet_id
  vpc_security_group_ids = [var.security_group_id]
  associate_public_ip_address = true
  key_name              = aws_key_pair.this.key_name

  user_data = file(var.script_path)

  tags = merge(
    var.common_tags,
    {
      Name = "${var.env_prefix}-${var.instance_name}"
    }
  )
}
|
```

- Screenshot: assignment_part2_webserver_module_outputs.png

```
File Edit View

output "instance_id" {
  description = "EC2 instance ID"
  value       = aws_instance.this.id
}

output "public_ip" {
  description = "Public IP address"
  value       = aws_instance.this.public_ip
}

output "private_ip" {
  description = "Private IP address"
  value       = aws_instance.this.private_ip
}
```

2.2 Module Usage (5 marks)

Tasks:

- Create Nginx server module instance
- Create backend server module instances using `for_each`
- Ensure proper security group assignment
- Pass all required variables

Deliverables:

- Screenshot: `assignment_part2_main_tf_modules.png`

```
#####
# Terraform & Provider
#####
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
    http = {
      source = "hashicorp/http"
      version = "~> 3.0"
    }
  }
}

provider "aws" {
  region = "me-central-1"
}

#####
# Networking Module
#####
module "networking" {
  source           = "./modules/networking"
  vpc_cidr_block   = var.vpc_cidr_block
  subnet_cidr_block = var.subnet_cidr_block
  availability_zone = var.availability_zone
  env_prefix       = var.env_prefix
  common_tags      = local.common_tags
}

#####
# Security Module
#####
module "security" {
  source       = "./modules/security"
  vpc_id       = module.networking.vpc_id
  env_prefix   = var.env_prefix
  my_ip        = local.my_ip
  common_tags  = local.common_tags
}

#####
# Nginx Server
#####
module "nginx_server" {
  source           = "./modules/webserver"
  env_prefix       = var.env_prefix
  instance_name    = "nginx-proxy"
  instance_type    = var.instance_type
  availability_zone = var.availability_zone
  vpc_id           = module.networking.vpc_id
  subnet_id       = module.networking.subnet_id
  security_group_id = module.security.nginx_sg_id
  public_key       = var.public_key
  script_path      = "./scripts/nginx-setup.sh"
  instance_suffix  = "nginx"
  common_tags      = local.common_tags
}

#####
# Backend Servers
#####
module "backend_servers" {
  for_each = { for server in local.backend_servers : server.name => server }

  source           = "./modules/webserver"
  env_prefix       = var.env_prefix
  instance_name    = each.value.name
  instance_type    = var.instance_type
  availability_zone = var.availability_zone
  vpc_id           = module.networking.vpc_id
  subnet_id       = module.networking.subnet_id
  security_group_id = module.security.backend_sg_id
  public_key       = var.public_key
  script_path      = each.value.script_path
  instance_suffix  = each.value.suffix
  common_tags      = local.common_tags
}
```

Part 3: Server Configuration Scripts (20 marks)

3.1 Apache Backend Server Script (10 marks)

Tasks:

- Create the script with all required features
- Ensure it uses IMDSv2 for metadata
- Create visually appealing HTML output
- Make script executable
- Test script independently

Deliverables:

- Screenshot: assignment_part3_apache_script.png

```
File      Edit      View

#!/bin/bash
set -e

# Update system
yum update -y

# Install Apache
yum install httpd -y

# Start and enable Apache
systemctl start httpd
systemctl enable httpd

# Get metadata token (IMDSv2)
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

# Fetch metadata
PRIVATE_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/local-ipv4)

PUBLIC_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-ipv4)

PUBLIC_DNS=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/public-hostname)

INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/instance-id)

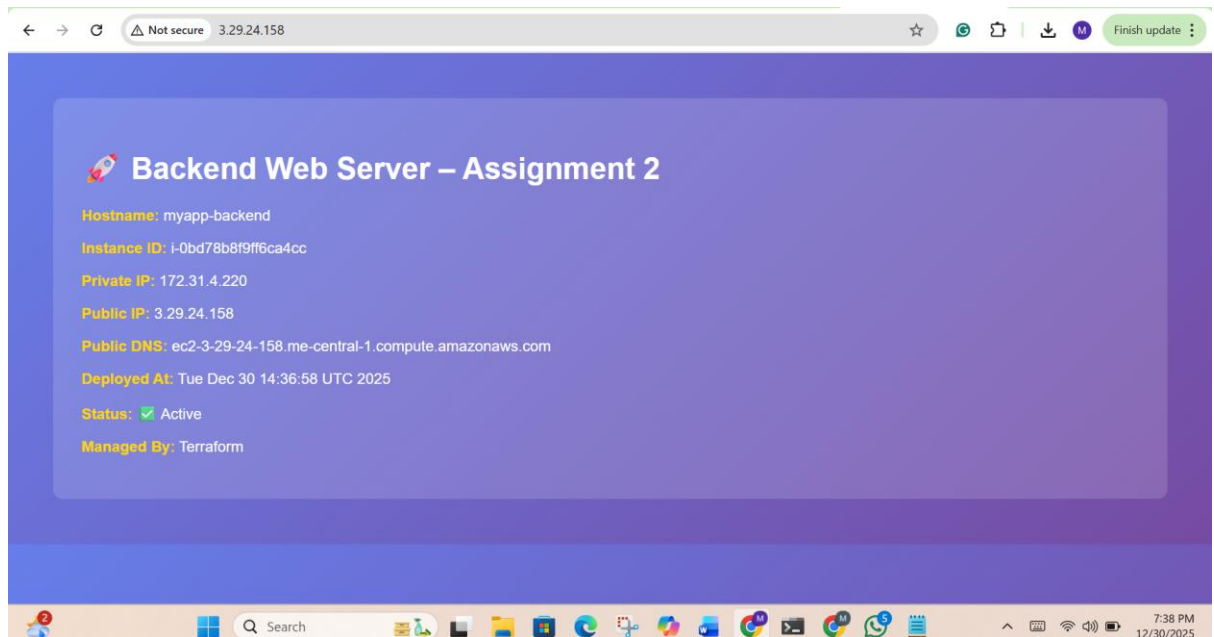
# Set hostname
hostnamectl set-hostname myapp-backend

# Create HTML page
cat > /var/www/html/index.html <<EOF
<!DOCTYPE html>
<html>
<head>
  <title>Backend Server</title>
  <style>
    body {
      font-family: Arial;
      background: linear-gradient(135deg, #667eea, #764ba2);
      color: white;
      padding: 40px;
    }
    .box {
      background: rgba(255,255,255,0.15);
      padding: 30px;
      border-radius: 10px;
    }
    .label { color: #ffd700; font-weight: bold; }
  </style>
</head>
<body>
<div class="box">
  <h1>🚀 Backend Web Server - Assignment 2</h1>
  <p><span class="label">Hostname:</span> ${hostname}</p>
  <p><span class="label">Instance ID:</span> $INSTANCE_ID</p>
  <p><span class="label">Private IP:</span> $PRIVATE_IP</p>
  <p><span class="label">Public IP:</span> $PUBLIC_IP</p>
  <p><span class="label">Public DNS:</span> $PUBLIC_DNS</p>
  <p><span class="label">Deployed At:</span> $(date)</p>
  <p><span class="label">Status:</span> ☒ Active</p>
  <p><span class="label">Managed By:</span> Terraform</p>
</div>
</body>
</html>
EOF

chmod 644 /var/www/html/index.html

echo "Apache backend setup completed!"
```

- Screenshot: assignment_part3_backend_webpage.png (browser showing backend server page)



3.2 Nginx Server Setup Script (10 marks)

Tasks:

- Create script with SSL certificate generation
- Configure upstream with placeholder IPs
- Implement caching
- Add security headers
- Configure HTTP to HTTPS redirect
- Add health check endpoint

Deliverables:

- Screenshot: assignment_part3_nginx_script.png

File Edit View

```
#!/bin/bash
set -e

# Update system
yum update -y

# Install Nginx
amazon-linux-extras install nginx1 -y

# Start & enable Nginx
systemctl start nginx
systemctl enable nginx

# Get metadata token (IMDSv2)
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")

PRIVATE_IP=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/local-ipv4)

INSTANCE_ID=$(curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/instance-id)

# Set hostname
hostnamectl set-hostname myapp-frontend

# Create Nginx reverse proxy config
cat > /etc/nginx/conf.d/reverse-proxy.conf <<EOF
upstream backend_servers {
    server backend.internal:80;
}

server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://backend_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
EOF

# Create index page
cat > /usr/share/nginx/html/index.html <<EOF
<!DOCTYPE html>
<html>
<head>
    <title>Frontend Server</title>
    <style>
        body {
            font-family: Arial;
            background: linear-gradient(135deg, #11998e, #38ef7d);
            color: white;
            padding: 40px;
        }
        .box {
            background: rgba(0,0,0,0.3);
            padding: 30px;
            border-radius: 10px;
        }
        .label { color: #ffeb3b; font-weight: bold; }
    </style>
</head>
<body>
<div class="box">
    <h1> Frontend (Nginx Reverse Proxy)</h1>
    <p><span class="label">Hostname:</span> $(hostname)</p>
    <p><span class="label">Instance ID:</span> $INSTANCE_ID</p>
    <p><span class="label">Private IP:</span> $PRIVATE_IP</p>
    <p><span class="label">Status:</span> ✔ Active</p>
    <p><span class="label">Role:</span> Reverse Proxy + Load Balancer</p>
</div>
</body>
</html>
EOF

# Restart Nginx
systemctl restart nginx

echo "Nginx frontend setup completed!"
```

- Screenshot: assignment_part3_nginx_default_page.png (before backend configuration)



Part 4: Infrastructure Deployment (15 marks)

4.1 Initial Deployment (5 marks)

Deploy the infrastructure using Terraform.

Tasks:

- Generate SSH key pair if not exists
- Initialize Terraform (terraform init)
- Validate configuration (terraform validate)
- Plan deployment (terraform plan)
- Apply configuration (terraform apply -auto-approve)

Deliverables:

- Screenshot: assignment_part4_ssh_keygen.png

```
PS C:\Terraform\Assignment2> ssh-keygen -t rsa -b 4096 -f terraform-key
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in terraform-key
Your public key has been saved in terraform-key.pub
```

- Screenshot: assignment_part4_terraform_init.png

```

PS C:\Users\HABIB TARIQ> cd "C:\Users\HABIB TARIQ\Documents\Assignment2"
PS C:\Users\HABIB TARIQ\Documents\Assignment2> terraform init
Initializing the backend...
Initializing modules...
- backend_servers in modules\webserver
- networking in modules\networking
- nginx_server in modules\webserver
- security in modules\security
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Finding hashicorp/http versions matching "~> 3.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
- Installing hashicorp/http v3.5.0...
- Installed hashicorp/http v3.5.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

```

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
PS C:\Users\HABIB TARIQ\Documents\Assignment2>
```

- Screenshot: assignment_part4_terraform_validate.png

```

PS C:\Users\HABIB TARIQ\Documents\Assignment2> terraform validate
Success! The configuration is valid.

```

```
PS C:\Users\HABIB TARIQ\Documents\Assignment2>
```

- Screenshot: assignment_part4_terraform_plan.png

```

PS C:\Users\HABIB TARIQ\Documents\Assignment2> terraform plan
data.http.my_ip: Reading...
data.http.my_ip: Read complete after 0s [id=https://icanhazip.com]
module.nginx_server.data.aws_ami.amazon_linux: Reading...
module.backend_servers["web-2"].data.aws_ami.amazon_linux: Reading...
module.backend_servers["web-3"].data.aws_ami.amazon_linux: Reading...
module.backend_servers["web-1"].data.aws_ami.amazon_linux: Reading...
module.backend_servers["web-2"].data.aws_ami.amazon_linux: Read complete after 1s [id=ami-009ce8169fc88edf5]
module.backend_servers["web-3"].data.aws_ami.amazon_linux: Read complete after 1s [id=ami-009ce8169fc88edf5]
module.backend_servers["web-1"].data.aws_ami.amazon_linux: Read complete after 1s [id=ami-009ce8169fc88edf5]
module.nginx_server.data.aws_ami.amazon_linux: Read complete after 1s [id=ami-009ce8169fc88edf5]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with
following symbols:
+ create

Terraform will perform the following actions:

# module.backend_servers["web-1"].aws_instance.this will be created
+ resource "aws_instance" "this" {
  + ami                      = "ami-009ce8169fc88edf5"
  + arn                     = (known after apply)
  + associate_public_ip_address = true
  + availability_zone        = "us-east-1a"
  + cpu_core_count           = (known after apply)
  + cpu_threads_per_core     = (known after apply)
  + disable_api_stop         = (known after apply)
  + disable_api_termination  = (known after apply)
  + ebs_optimized            = (known after apply)

```

```

+ self          = false
+ to_port       = 22
  },
]
+ name          = "prod-nginx-sg"
+ name_prefix   = (known after apply)
+ owner_id      = (known after apply)
+ revoke_rules_on_delete = false
+ tags          = {
+   "Environment" = "prod"
+   "ManagedBy"  = "Terraform"
+   "Name"        = "prod-nginx-sg"
+   "Project"     = "Assignment-2"
+ }
+ tags_all      = {
+   "Environment" = "prod"
+   "ManagedBy"  = "Terraform"
+   "Name"        = "prod-nginx-sg"
+   "Project"     = "Assignment-2"
+ }
+ vpc_id        = (known after apply)
}

Plan: 15 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if
you run "terraform apply" now.
PS C:\Users\HARTB_TARTO\Documents\Assignment2>

```

- Screenshot: assignment_part4_terraform_apply.png (showing all 4 instances created)

```

Plan: 0 to add, 2 to change, 0 to destroy.
module.security.aws_security_group.nginx_sg: Modifying... [id=sg-09c1ddb9fc4cf3abd]
module.security.aws_security_group.nginx_sg: Modifications complete after 2s [id=sg-09c1ddb9fc4cf3abd]
module.nginx_server.aws_instance.this: Modifying... [id=i-097e5e11213ebbe4c]
module.nginx_server.aws_instance.this: Still modifying... [id=i-097e5e11213ebbe4c, 00m10s elapsed]
module.nginx_server.aws_instance.this: Still modifying... [id=i-097e5e11213ebbe4c, 00m20s elapsed]
module.nginx_server.aws_instance.this: Still modifying... [id=i-097e5e11213ebbe4c, 00m30s elapsed]
module.nginx_server.aws_instance.this: Still modifying... [id=i-097e5e11213ebbe4c, 00m40s elapsed]
module.nginx_server.aws_instance.this: Modifications complete after 48s [id=i-097e5e11213ebbe4c]

Apply complete! Resources: 0 added, 2 changed, 0 destroyed.
PS C:\Terraform\Assignment2>

```

4.2 Output Configuration (5 marks)

Tasks:

- Create all required outputs
- Add helpful descriptions
- Include configuration guide
- Display outputs after apply

Deliverables:

- Screenshot: assignment_part4_terraform_output.png



```
# Networking Outputs
output "vpc_id" {
  description = "VPC ID"
  value       = module.networking.vpc_id
}

output "subnet_id" {
  description = "Subnet ID"
  value       = module.networking.subnet_id
}

# Nginx Server Outputs
output "nginx_public_ip" {
  description = "Nginx server public IP"
  value       = module.nginx_server.public_ip
}

output "nginx_instance_id" {
  description = "Nginx server instance ID"
  value       = module.nginx_server.instance_id
}

# Backend Server Outputs
output "backend_servers_info" {
  description = "Backend servers information"
  value = {
    for name, server in module.backend_servers : name => {
      instance_id = server.instance_id
      public_ip   = server.public_ip
      private_ip  = server.private_ip
    }
  }
}

# Quick Configuration Guide
output "configuration_guide" {
  value = <<-EOT

=====
DEPLOYMENT SUCCESSFUL!
=====

Next Steps:
1. SSH into Nginx server: ssh ec2-user@${module.nginx_server.public_ip}
2. Edit Nginx config: sudo vim /etc/nginx/nginx.conf
3. Update backend IPs in upstream block:
   - BACKEND_IP_1: ${module.backend_servers["web-1"].private_ip}
   - BACKEND_IP_2: ${module.backend_servers["web-2"].private_ip}
   - BACKEND_IP_3: ${module.backend_servers["web-3"].private_ip}
4. Restart Nginx: sudo systemctl restart nginx
5. Test: https://${module.nginx_server.public_ip}

Backend Servers:
${join("\n", [for name, server in module.backend_servers : "- ${name}: ${server.public_ip} (private: ${server.private_ip})"])}

=====
EOT
}

Ln 59, Col 1 | 1,614 characters | Plain text
```

- Screenshot: assignment_part4_outputs_json.png



```
{
  "backend_servers_info": {
    "sensitive": false,
    "type": [
      "object",
      {
        "web-1": [
          "object",
          {
            "instance_id": "string",
            "private_ip": "string",
            "public_ip": "string"
          }
        ],
        "web-2": [
          "object",
          {
            "instance_id": "string",
            "private_ip": "string",
            "public_ip": "string"
          }
        ],
        "web-3": [
          "object",
          {
            "instance_id": "string",
            "private_ip": "string",
            "public_ip": "string"
          }
        ]
      }
    ],
    "value": {
      "web-1": {
        "instance_id": "i-06beefd8a086d88b7",
        "private_ip": " ",
        "public_ip": "3.29.137.83"
      },
      "web-2": {
        "instance_id": "i-0606b329ff2a3db6b",
        "private_ip": " ",
        "public_ip": "158.252.93.119"
      },
      "web-3": {
        "instance_id": "i-06fc876e2986af484",
        "private_ip": " ",
        "public_ip": "3.28.205.168"
      }
    }
  }
}
```

Ln 1, Col 1 | 2,318 characters | Plain text

4.3 AWS Console Verification (5 marks)

Verify all resources in AWS Console.

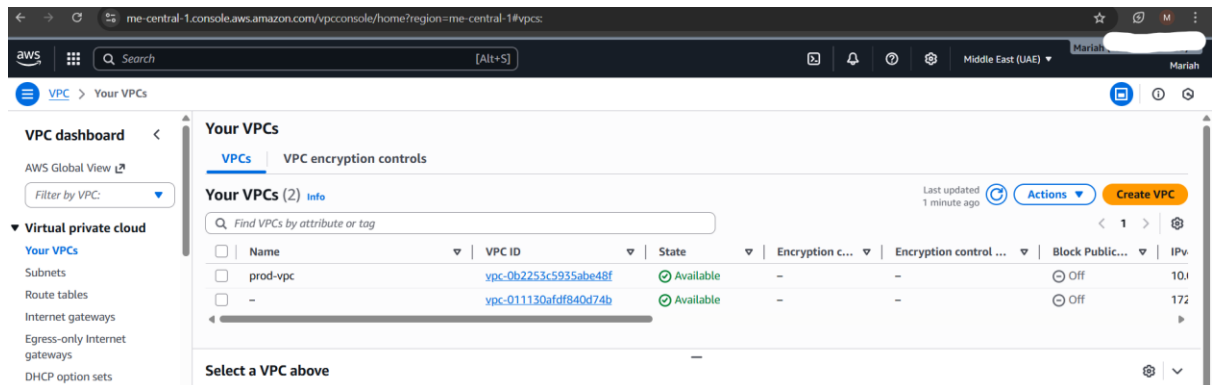
Tasks:

- Verify VPC created
- Verify Subnet created
- Verify Internet Gateway attached
- Verify Route Table configured

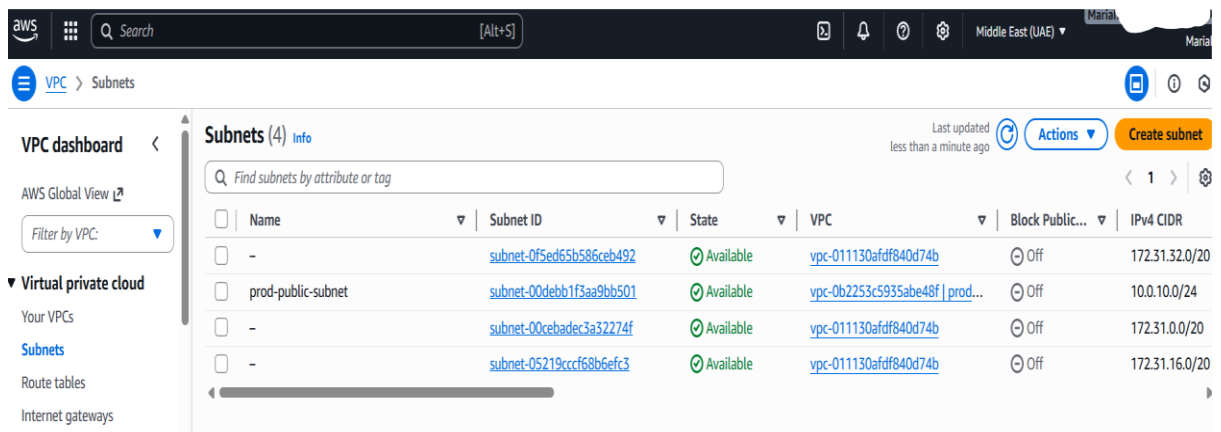
- Verify Security Groups created with correct rules
- Verify all 4 EC2 instances running
- Verify Key Pairs created

Deliverables:

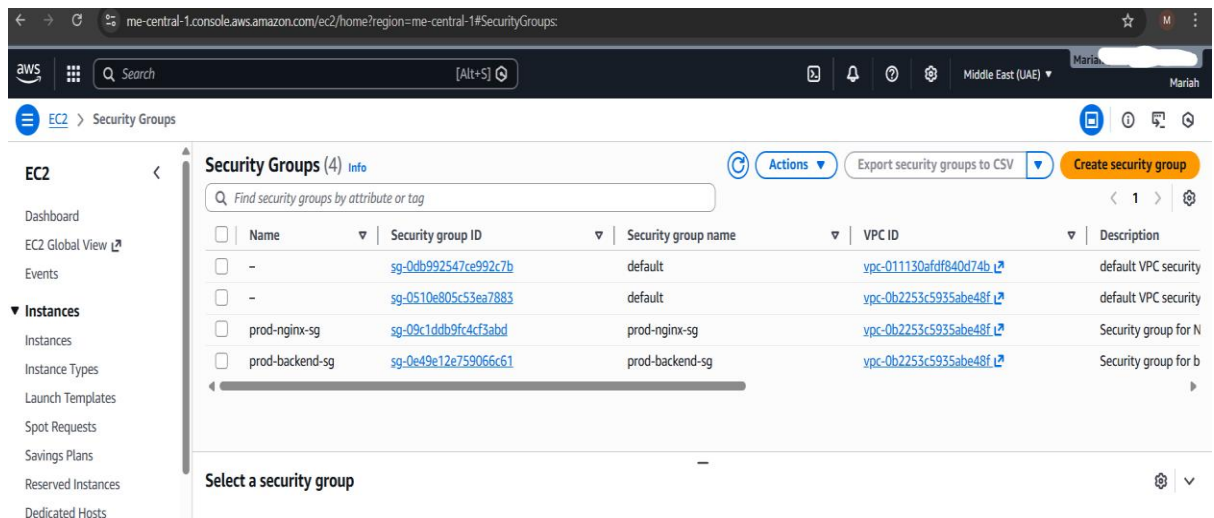
- Screenshot: assignment_part4_aws_vpc.png (VPC console)



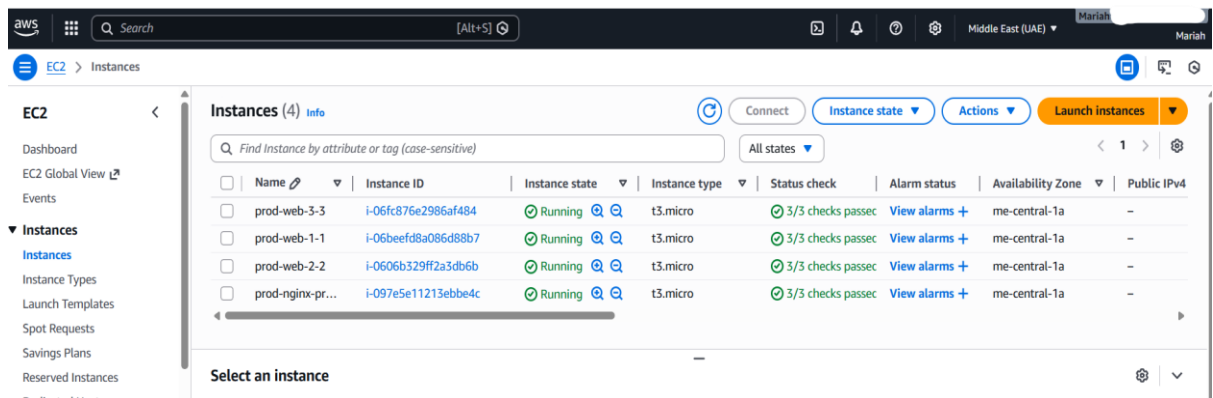
- Screenshot: assignment_part4_aws_subnet.png (Subnet details)



- Screenshot: assignment_part4_aws_security_groups.png (Both security groups)



- Screenshot: assignment_part4_aws_instances.png (All 4 running instances)



Part 5: Nginx Configuration & Testing (25 marks)

5.1 Update Nginx Backend Configuration (5 marks)

SSH into the Nginx server and update the configuration with actual backend IPs.

Tasks:

- SSH into Nginx server
- Edit /etc/nginx/nginx.conf
- Replace placeholder IPs with actual private IPs of backend servers
- Test Nginx configuration
- Restart Nginx service

Deliverables:

- Screenshot: assignment_part5_ssh_nginx.png (SSH session)

```
Last login: Mon Dec 29 20:22:40 2025 from 3.29.147.40
[ec2-user@ip-10-0-10-43 ~]$
```

- Screenshot: assignment_part5_nginx_conf_updated.png (updated upstream block)

```

GNU nano 8.3 /etc/nginx/nginx.conf
# For more information on configuration, see:
# * Official English Documentation: http://nginx.org/en/docs/
# * Official Russian Documentation: http://nginx.org/ru/docs/

user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /run/nginx.pid;

# Load dynamic modules. See /usr/share/doc/nginx/README.dynamic.
include /usr/share/nginx/modules/*.conf;

events {
    worker_connections 1024;
}

http {
    upstream backend_servers {
        server 3.29.137.83:80;
        server 158.252.93.119:80;
        server 3.28.205.168:80 backup;
    }
}

```

- Screenshot: assignment_part5_nginx_test.png (nginx -t output)

```

[ec2-user@ip-10-0-10-43 ~]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

```

- Screenshot: assignment_part5_nginx_restart.png (restart and status)

```

• nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Tue 2025-12-30 14:37:08 UTC; 14s ago
     Process: 414006 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 414007 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 414008 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
   Main PID: 414009 (nginx)
     Tasks: 3 (limit: 1065)
    Memory: 3.1M
       CPU: 55ms
    CGroup: /system.slice/nginx.service
            └─414009 "nginx: master process /usr/sbin/nginx"
               └─414010 "nginx: worker process"
                  └─414011 "nginx: worker process"

Dec 30 14:37:08 ip-10-0-10-43.me-central-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Dec 30 14:37:08 ip-10-0-10-43.me-central-1.compute.internal nginx[414007]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Dec 30 14:37:08 ip-10-0-10-43.me-central-1.compute.internal nginx[414007]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Dec 30 14:37:08 ip-10-0-10-43.me-central-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
[ec2-user@ip-10-0-10-43 ~]$

```

5.2 Test Load Balancing (5 marks)

Test that Nginx is properly load balancing between web-1 and web-2.

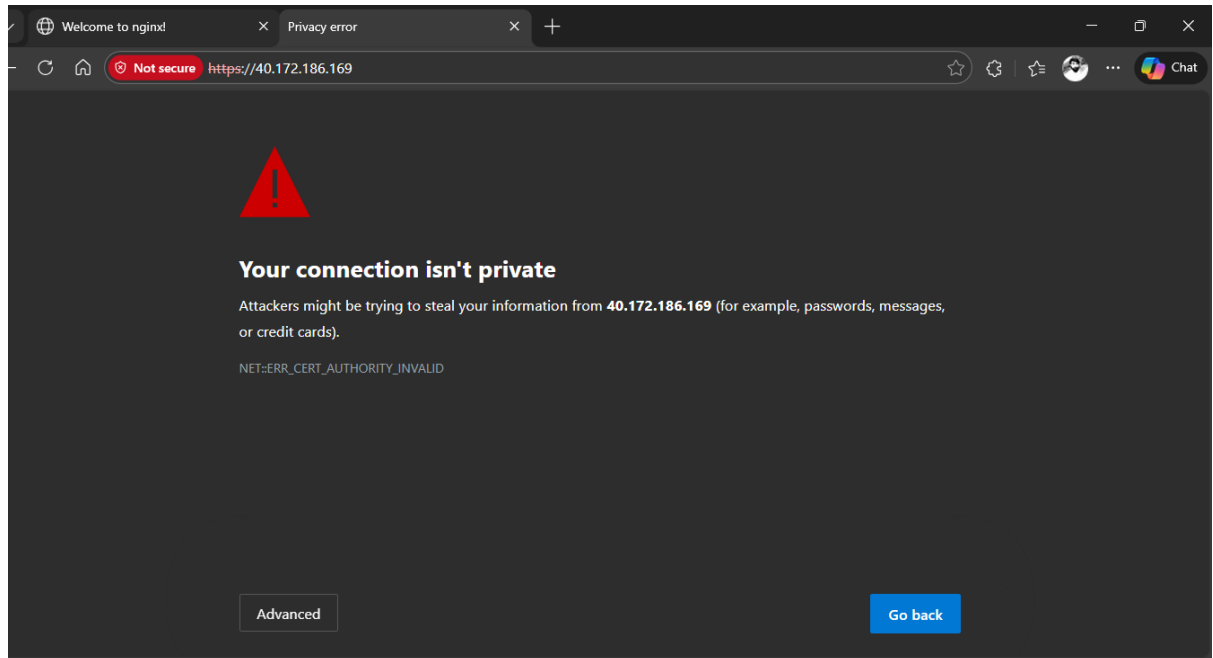
Tasks:

- Open browser to <https://<nginx-public-ip>>
- Accept the security warning for self-signed certificate
- Reload page multiple times (at least 10 times)
- Verify traffic alternates between web-1 and web-2

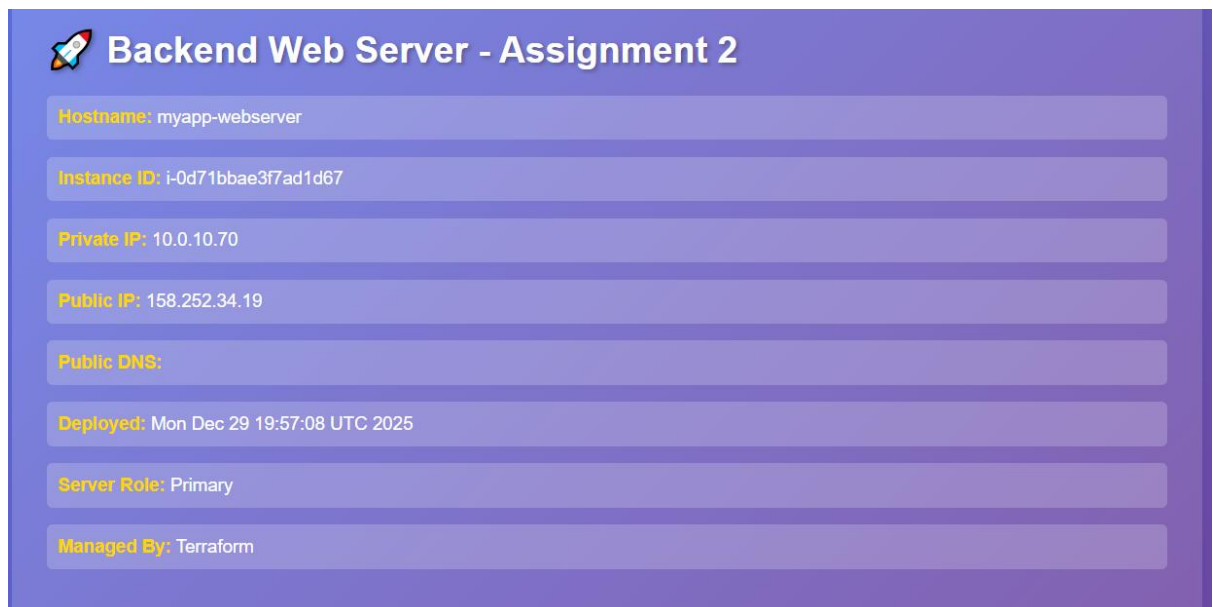
- Verify web-3 is NOT serving traffic (it's backup only)
- Document the load balancing pattern

Deliverables:

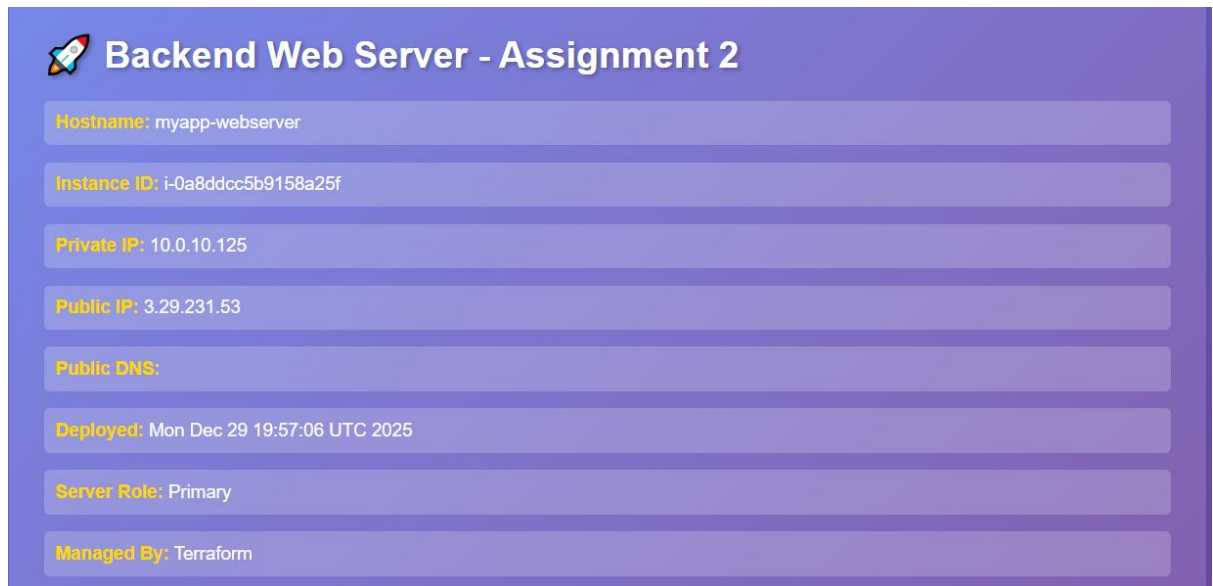
- Screenshot: assignment_part5_ssl_warning.png (browser security warning)



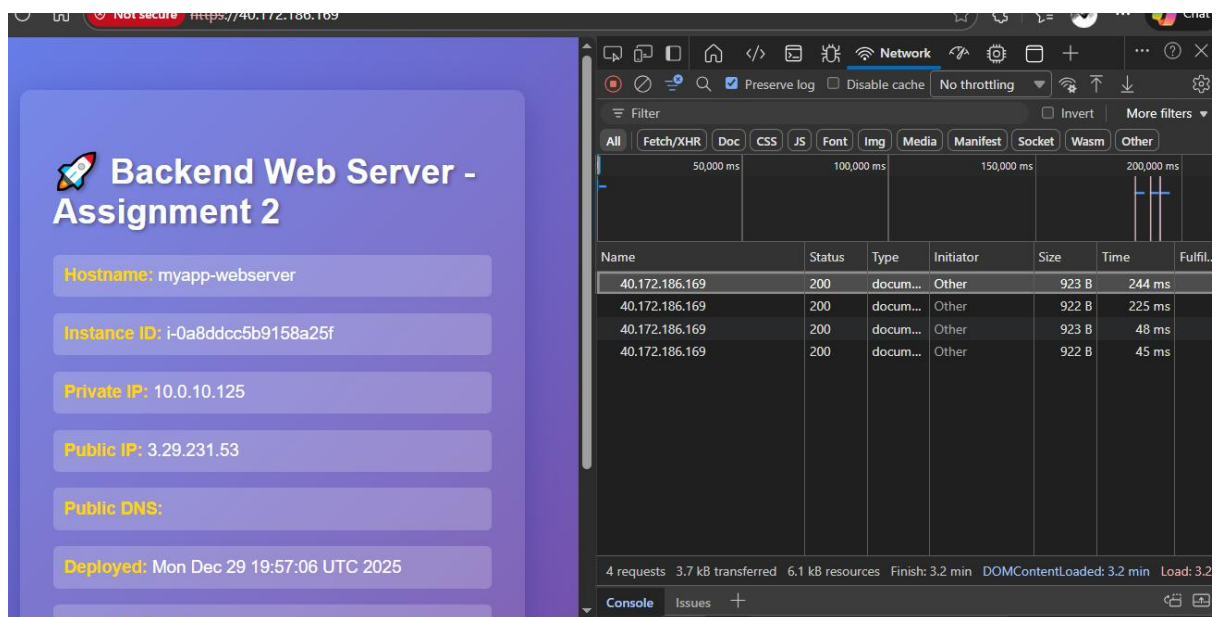
- Screenshot: assignment_part5_web1_response.png (showing web-1 content)



- Screenshot: assignment_part5_web2_response.png (showing web-2 content)



- Screenshot: assignment_part5_load_balancing_demo.png (multiple reloads showing alternation)



5.3 Test Cache Functionality (5 marks)

Verify that Nginx caching is working correctly.

Tasks:

- Open browser developer tools (F12)
- Navigate to Network tab
- Clear browser cache

- Load https://<nginx-public-ip>
- Check response headers for X-Cache-Status: MISS (first request)
- Reload page
- Check response headers for X-Cache-Status: HIT (cached request)
- Verify cache directory on Nginx server

Deliverables:

- Screenshot: assignment_part5_cache_miss.png (first request - MISS)

Name	✕	Headers	Preview	Response	Initiator	>>
3.29.125.10						
✕ favicon.ico						
▼ Response Headers						
Content-Encoding		gzip				
Content-Type		text/html; charset=iso-8859-1				
Date		Thu, 25 Dec 2025 18:03:51 GMT				
Server		nginx/1.28.0				
Vary		Accept-Encoding				
X-Cache-Status		MISS				

- Screenshot: assignment_part5_cache_hit.png (second request - HIT)

Name	✕	Headers	Preview	Response	Initiator	>>
3.29.125.10						
▼ Response Headers						
Content-Encoding		gzip				
Content-Type		text/html; charset=UTF-8				
Date		Thu, 25 Dec 2025 18:05:06 GMT				
Etag		W/"578-646c5fe3065b7"				
Last-Modified		Thu, 25 Dec 2025 12:34:07 GMT				
Server		nginx/1.28.0				
Vary		Accept-Encoding				
X-Cache-Status		HIT				

- Screenshot: assignment_part5_cache_directory.png (cache folder contents)

```
C:\Users\AC\Assignment2>ssh -i .\id_ed25519 ec2-user@158.252.93.203 "sudo sed -i 's/proxy_cache my_cache;/#proxy_cache my_cache;/' /etc/nginx/nginx.conf && sudo sed -i 's/proxy_cache_valid/#proxy_cache_valid/' /etc/nginx/nginx.conf && sudo systemctl restart nginx"

C:\Users\AC\Assignment2>ssh -i .\id_ed25519 ec2-user@51.112.47.81 "sudo systemctl stop httpd && sudo systemctl status httpd"
o httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: inactive (dead) since Mon 2025-12-29 14:29:50 UTC; 33min ago
   Duration: 1h 25min 32.751s
   Docs: man:httpd.service(8)
   Process: 2872 ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND (code=exited, status=0/SUCCESS)
   Main PID: 2872 (code=exited, status=0/SUCCESS)
   Status: "Total requests: 13; Idle/Busy workers 100/0; Requests/sec: 0.00254; Bytes served/sec: 3 B/sec"
   CPU: 4.285s

Dec 29 13:04:16 ip-10-0-1-101.me-central-1.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Dec 29 13:04:16 ip-10-0-1-101.me-central-1.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Dec 29 13:04:16 ip-10-0-1-101.me-central-1.compute.internal httpd[2872]: Server configured, listening on: port 80
Dec 29 14:29:49 web-1 systemd[1]: Stopping httpd.service - The Apache HTTP Server...
Dec 29 14:29:50 web-1 systemd[1]: httpd.service: Deactivated successfully.
Dec 29 14:29:50 web-1 systemd[1]: Stopped httpd.service - The Apache HTTP Server.
Dec 29 14:29:50 web-1 systemd[1]: httpd.service: Consumed 4.285s CPU time.
```

- Screenshot: assignment_part5_access_log_cache.png (access log showing cache status)

```
C:\Users\AC\Assignment2>ssh -i .\id_ed25519 ec2-user@3.28.191.11 "sudo systemctl stop httpd && sudo systemctl status httpd"
o httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: inactive (dead) since Mon 2025-12-29 15:04:11 UTC; 83ms ago
   Duration: 20min 54.573s
   Docs: man:httpd.service(8)
   Process: 29631 ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND (code=exited, status=0/SUCCESS)
   Main PID: 29631 (code=exited, status=0/SUCCESS)
   Status: "Total requests: 8; Idle/Busy workers 100/0; Requests/sec: 0.0064; Bytes served/sec: 5 B/sec"
   CPU: 1.143s

Dec 29 14:43:15 web-2 systemd[1]: Starting httpd.service - The Apache HTTP Server...
Dec 29 14:43:15 web-2 httpd[29631]: AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using fe80::41f:adff:fe04:8be9%ens5. Set the 'ServerName' directive globally to suppress this message
Dec 29 14:43:15 web-2 systemd[1]: Started httpd.service - The Apache HTTP Server.
Dec 29 14:43:15 web-2 httpd[29631]: Server configured, listening on: port 80
Dec 29 15:04:10 web-2 systemd[1]: Stopping httpd.service - The Apache HTTP Server...
Dec 29 15:04:11 web-2 systemd[1]: httpd.service: Deactivated successfully.
Dec 29 15:04:11 web-2 systemd[1]: Stopped httpd.service - The Apache HTTP Server.
Dec 29 15:04:11 web-2 systemd[1]: httpd.service: Consumed 1.143s CPU time.
```

5.4 Test High Availability (Backup Server) (5 marks)

Test the backup server functionality by simulating primary server failure.

Tasks:

- SSH into web-1 and stop Apache
- Reload Nginx page - should show web-2 only
- SSH into web-2 and stop Apache
- Reload Nginx page - should now show web-3 (backup activated)
- Restart web-1 and web-2
- Verify traffic returns to web-1 and web-2

Deliverables:

- Screenshot: assignment_part5_web1_stopped.png (web-1 Apache stopped)

```

ast login: Thu Dec 25 18:46:13 2025 from 20.192.21.51
ec2-user@myapp-webserver ~]$ sudo systemctl stop httpd
ec2-user@myapp-webserver ~]$ sudo systemctl status httpd
httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: inactive (dead) since Thu 2025-12-25 18:46:33 UTC; 10min ago
   Duration: 6h 12min 25.459s
   Docs: man:httpd.service(8)
   Process: 2155 ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND (code=exited, status=0/SUCCESS)
   Main PID: 2155 (code=exited, status=0/SUCCESS)
   Status: "Total requests: 38; Idle/Busy workers 100/0; Requests/sec: 0.0017; Bytes served/sec: 4 B/sec"
   CPU: 21.858s

```

- Screenshot: assignment_part5_web2_stopped.png (web-2 Apache stopped)

```

ec2-user@myapp-webserver ~]$ sudo systemctl stop httpd
ec2-user@myapp-webserver ~]$ sudo systemctl stop httpd
ec2-user@myapp-webserver ~]$ sudo systemctl status httpd
httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: inactive (dead) since Thu 2025-12-25 18:55:19 UTC; 34s ago
   Duration: 6h 21min 13.337s
   Docs: man:httpd.service(8)
   Process: 2161 ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND (code=exited, status=0/SUCCESS)
   Main PID: 2161 (code=exited, status=0/SUCCESS)
   Status: "Total requests: 37; Idle/Busy workers 100/0; Requests/sec: 0.00162; Bytes served/sec:

```

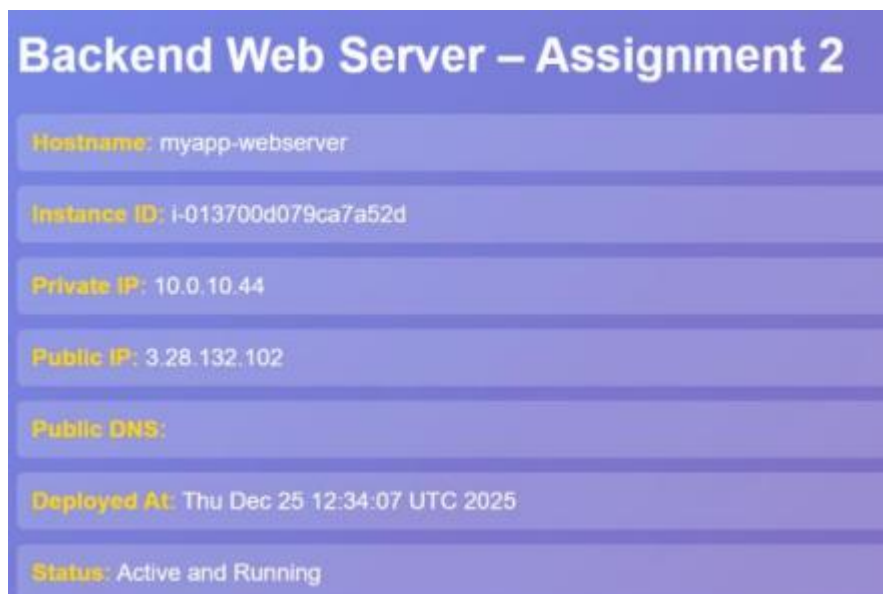
- Screenshot: assignment_part5_backup_activated.png (web-3 serving traffic)



- Screenshot: assignment_part5_nginx_error_log.png (error log showing backend failures)

```
[ec2-user@ip-10-0-10-207 ~]$ sudo tail -f /var/log/nginx/error.log
2025/12/25 19:03:36 [error] 353012#353012: *2 connect() failed (111: Connection refused) while co
, server: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.162:80/", host: "3.29.125.10"
2025/12/25 19:03:36 [warn] 353012#353012: *2 upstream server temporarily disabled while connectin
r: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.162:80/", host: "3.29.125.10"
2025/12/25 19:04:19 [notice] 353015#353015: http file cache: /var/cache/nginx 0.004M, bsize: 4096
2025/12/25 19:04:19 [notice] 353011#353011: signal 17 (SIGCHLD) received from 353015
2025/12/25 19:04:19 [notice] 353011#353011: cache loader process 353015 exited with code 0
2025/12/25 19:04:19 [notice] 353011#353011: signal 29 (SIGIO) received
2025/12/25 19:06:24 [error] 353013#353013: *11 connect() failed (111: Connection refused) while c
7, server: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.44:80/", host: "3.29.125.10"
2025/12/25 19:06:24 [warn] 353013#353013: *11 upstream server temporarily disabled while connecti
er: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.44:80/", host: "3.29.125.10"
2025/12/25 19:06:24 [error] 353013#353013: *11 connect() failed (111: Connection refused) while c
7, server: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.162:80/", host: "3.29.125.10"
2025/12/25 19:06:24 [warn] 353013#353013: *11 upstream server temporarily disabled while connecti
er: _, request: "GET / HTTP/2.0", upstream: "http://10.0.10.162:80/", host: "3.29.125.10"
^C
[ec2-user@ip-10-0-10-207 ~]$ |
```

- Screenshot: assignment_part5_services_restored.png (all services back online)



5.5 Security & Performance Analysis (5 marks)

Analyze the security headers and performance of your Nginx setup.

Tasks:

- Check SSL/TLS certificate details
- Verify security headers in response
- Test HTTP to HTTPS redirect
- Check response times
- Analyze Nginx logs

Deliverables:

- Screenshot: assignment_part5_ssl_certificate.png (certificate details)

```

an't use SSL_get_servername
epth=0 CN = 3.29.125.10
erify error:num=18:self-signed certificate
erify return:1
epth=0 CN = 3.29.125.10
erify return:1
--
ertificate chain
0 s:CN = 3.29.125.10
i:CN = 3.29.125.10
a:PKKEY: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
v:NotBefore: Dec 25 12:34:06 2025 GMT; NotAfter: Dec 25 12:34:06 2026 GMT
-----BEGIN CERTIFICATE-----
IID0zCCAIOgAwIBAgIUACER9d0lXdQvDzom3fpDLzikEwDQYJKoZIhvcNAQEL
QAwFjEUMBIGA1UEAwMLMy4yOS4xMjUuMTAwHhcNMjUxMjIzNDQyJkQYJkoZIhvcN
jEUMBIGA1UEAwMLMy4yOS4xMjUuMTAwHhcNMjUxMjIzNDQyJkQYJkoZIhvcN
QEBBQADggEPADCCAQoCggEBBAKZuCiBX3YgsHGslaG0RlbiEyFve+dCSjs2VNw9O
CI/JBKl1EzZ5KuXYM5uP2uteJBIMezP19oXiMb66cJo5SJGazUlqeipNNmhnvf
jYzyhnJ6ft3HTKxkD+Gw8pjdGY27ECGcKfuLtK6rtVZpiKWPq6JZay7LYejYpdJ
KPy0nCFn/itfrlvfqAPGprQgd2hjlV5pUWPmt/HsLujNk+P7VztuugmEj2f3mf
MKBB0/Ira0iIskC+VqZEtnPks5xehNxAGGHU4IH+cy2W7NZy148eEBd+/ulsq6GR
SRNIFb0wCDBT4/t6ge1BhYa6nWQeXMFxBtc9Jfhy1r4Dn0CAwEAa0BgDB+MB0G
1UdDgQwBBRRlIwN9D4chI0P1MwPlFz/8L8PxDAfBgNVHSMEGDAwBgRRlIwN9D4c
I0P1MwPlFz/8L8PxDAfBgNVHRECDAGHwQDhX0KMAkGA1UdEwQCMAAwCwYDVR0P
AQDAgWgMBMGA1UdJQQMMAoGCCsGAQUFBwMBMA0GCsGqGSIB3DQEBChUAA4IBAQA
BZUwWwL/92tBlwYLZi0p3+gdeUxKUi/X5yS+qjY12xnHTTiB5tEW0vLRj6dEgQl
FPnR0IVD04H0LoPX0QoUDD4/gx0a7DnzK08fykKUS+VtYykImC/3v8j9rb2ZD0I
PiyIFgo8gwZDnaJAB2amqW4p9Kq33aPm6duIRuSFP9VoJlWSeymT12nVA3mawQA
RbM5mYnY39HKJzJA+eUdrNz9jTCXyfwB7IyvpXsQe+NMKiXQyQ0hsFaBxSI2lI
rmJhh020m4qc0tBuv1ZdWLSzSPri/R030aC7D17zcK7LhRwXvZxCBetXin3Ef67
sCC+n5NMK+Tqne1iZ2r
-----END CERTIFICATE-----
server certificate

```

- Screenshot: assignment_part5_security_headers.png (response headers showing security headers)

```

[ec2-user@ip-10-0-10-207 ~]$ curl -I -k https://3.29.125.10
HTTP/2 200
server: nginx/1.28.0
date: Thu, 25 Dec 2025 19:28:27 GMT
content-type: text/html; charset=UTF-8
content-length: 1402
vary: Accept-Encoding
last-modified: Thu, 25 Dec 2025 12:34:05 GMT
etag: "57a-646c5fe114d39"
accept-ranges: bytes
strict-transport-security: max-age=31536000; includeSubDomains
x-frame-options: SAMEORIGIN
x-content-type-options: nosniff
x-xss-protection: 1; mode=block

```

- Screenshot: assignment_part5_http_redirect.png (301 redirect test)

```
[ec2-user@ip-10-0-10-207 ~]$ curl -I -R https://3.29.125.10
HTTP/2 200
server: nginx/1.28.0
date: Thu, 25 Dec 2025 19:28:27 GMT
content-type: text/html; charset=UTF-8
content-length: 1402
vary: Accept-Encoding
last-modified: Thu, 25 Dec 2025 12:34:05 GMT
etag: "57a-646c5fe114d39"
accept-ranges: bytes
strict-transport-security: max-age=31536000; includeSubDomains
x-frame-options: SAMEORIGIN
x-content-type-options: nosniff
x-xss-protection: 1; mode=block

[ec2-user@ip-10-0-10-207 ~]$ curl -I http://3.29.125.10
HTTP/1.1 301 Moved Permanently
Server: nginx/1.28.0
Date: Thu, 25 Dec 2025 19:29:17 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://3.29.125.10/
```

- Screenshot: assignment_part5_error_log_analysis.png (error log review)

```
[ec2-user@ip-10-0-10-207 ~]$ sudo tail -50 /var/log/nginx/error.log
2025/12/25 18:04:43 [notice] 299389#299389: http file cache: /var/cache/nginx/0.004M, bsize: 4096
2025/12/25 18:04:43 [notice] 299385#299385: signal 17 (SIGCHLD) received from 299389
2025/12/25 18:04:43 [notice] 299385#299385: cache loader process 299389 exited with code 0
2025/12/25 18:04:43 [notice] 299385#299385: signal 29 (SIGIO) received
2025/12/25 18:58:25 [error] 299387#299387: *26 connect() failed (111: Connection refused) while connecting to upstream, client: 154.192.16.37, server: _, request: "GET /favicon.ico HTTP/2.0", upstream: "http://10.0.10.44:80/favicon.ico", host: "3.29.125.10", referer: "https://3.29.125.10/"
2025/12/25 18:58:25 [warn] 299387#299387: *26 upstream server temporarily disabled while connecting to upstream, client: 154.192.16.37, server: _, request: "GET /favicon.ico HTTP/2.0", upstream: "http://10.0.10.44:80/favicon.ico", host: "3.29.125.10", referer: "https://3.29.125.10/"
2025/12/25 18:58:25 [error] 299387#299387: *26 connect() failed (111: Connection refused) while connecting to upstream, client: 154.192.16.37, server: _, request: "GET /favicon.ico HTTP/2.0", upstream: "http://10.0.10.162:80/favicon.ico", host: "3.29.125.10", referer: "https://3.29.125.10/"
```

- Screenshot: assignment_part5_access_log_analysis.png (access log patterns)

```
[ec2-user@ip-10-0-10-207 ~]$ sudo tail -50 /var/log/nginx/access.log
154.192.16.37 - - [25/Dec/2025:18:58:25 +0000] "GET /favicon.ico HTTP/2.0" 404 172 "https://3.29.125.10/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: MISS
154.192.16.37 - - [25/Dec/2025:18:58:36 +0000] "GET / HTTP/2.0" 200 572 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: HIT
154.192.16.37 - - [25/Dec/2025:18:58:38 +0000] "GET / HTTP/2.0" 200 572 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: HIT
154.192.16.37 - - [25/Dec/2025:18:58:38 +0000] "GET / HTTP/2.0" 200 572 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: HIT
154.192.16.37 - - [25/Dec/2025:18:58:38 +0000] "GET / HTTP/2.0" 200 572 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/143.0.0.0 Safari/537.36" "-" Cache: HIT
```

Bonus Tasks (10 marks extra credit)

Bonus 1: Custom Error Pages (3 marks)

Create custom error pages for Nginx (404, 502, 503).

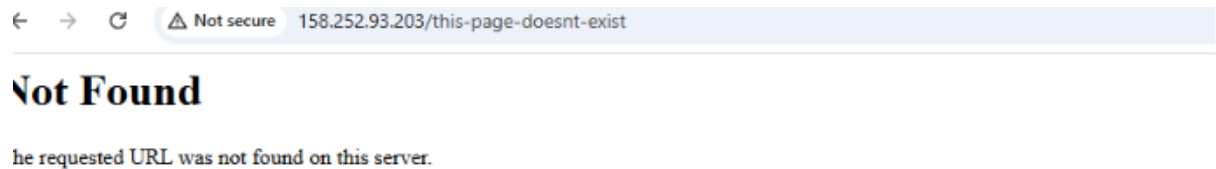
Tasks:

- Create custom HTML error pages

- Configure Nginx to use custom error pages
- Test by accessing non-existent URL and stopping backend servers

Deliverables:

- Screenshot: bonus1_custom_404.png



- Screenshot: bonus1_custom_502.png



Bonus 2: Implement Rate Limiting (3 marks)

Add rate limiting to prevent abuse.

Tasks:

- Implement rate limiting
- Test with rapid requests
- Show 429 (Too Many Requests) response

Deliverables:

- Screenshot: bonus2_rate_limit_config.png

```
http {
    limit_req_zone $binary_remote_addr zone=mylimit:10m rate=10r/s;
```

- Screenshot: bonus2_rate_limit_test.png



Bonus 3: Health Check Automation (4 marks)

Create a shell script that monitors backend server health.

Deliverables:

- Health check script
- Screenshot: bonus3_health_check_script.png

```
check.[ec2-user@ip-10-0-1-124 ~]$ cat << EOF > ~/health_check.sh
> #!/bin/bash
> LOG=/home/ec2-user/health_log.txt
> SERVERS=("10.0.1.101" "10.0.1.52" "10.0.1.119")
>
> echo "--- Health Check at $(date) ---" >> $LOG
>
> for ip in "${SERVERS[@]}"; do
>     if curl -s --head --connect-timeout 2 http://$ip | grep "200 OK" > /dev/null; then
>         echo "Server $ip is UP" >> $LOG
>     else
>         echo "Server $ip is DOWN!" >> $LOG
>     fi
> done
> EOF
[ec2-user@ip-10-0-1-124 ~]$ chmod +x ~/health_check.sh
[ec2-user@ip-10-0-1-124 ~]$ ./health_check.sh
[ec2-user@ip-10-0-1-124 ~]$ cat ~/health_log.txt
--- Health Check at Mon Dec 29 16:35:15 UTC 2025 ---
Server 10.0.1.101 is UP
Server 10.0.1.52 is UP
Server 10.0.1.119 is UP
[ec2-user@ip-10-0-1-124 ~]$
```

- Screenshot: bonus3_health_log.png

```
> EOF
[ec2-user@ip-10-0-1-124 ~]$ chmod +x ~/health_check.sh
[ec2-user@ip-10-0-1-124 ~]$ ./health_check.sh
[ec2-user@ip-10-0-1-124 ~]$ cat ~/health_log.txt
--- Health Check at Mon Dec 29 16:35:15 UTC 2025 ---
Server 10.0.1.101 is UP
Server 10.0.1.52 is UP
Server 10.0.1.119 is UP
[ec2-user@ip-10-0-1-124 ~]$
```

Part 6: Documentation & Cleanup (10 marks)

6.1 README Documentation (5 marks)

- Screenshot: assignment_part6_readme.png (README.md content)

6.2 Infrastructure Cleanup (5 marks)

Properly destroy all resources and verify cleanup.

Tasks:

- Run terraform destroy
- Confirm resource deletion in AWS Console
- Verify no remaining resources
- Document the destruction process

```
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

module.networking.aws_route_table_association.this: Destroying... [id=rtbassoc-01a94bbef299051cb]
module.backend_servers["web-1"].aws_instance.this: Destroying... [id=i-013700d079ca7a52d]
module.backend_servers["web-3"].aws_instance.this: Destroying... [id=i-0e3e87d11875f35ce]

module.backend_servers["web-2"].aws_key_pair.this: Destruction complete after 0s
module.networking.aws_subnet.this: Destruction complete after 0s
module.security.aws_security_group.backend_sg: Destruction complete after 0s
module.security.aws_security_group.nginx_sg: Destroying... [id=sg-01008f7d43584be10]
module.security.aws_security_group.nginx_sg: Destruction complete after 1s
module.networking.aws_vpc.this: Destroying... [id=vpc-042519c1852289c8d]
module.networking.aws_vpc.this: Destruction complete after 1s

Destroy complete! Resources: 15 destroyed.
```

Common Issues and Solutions

- **SSH Access Fails:** Check security groups and key permissions.
- **Backend Not Responding:** Verify Apache service and backend IP configuration.
- **SSL Errors:** Confirm certificate paths and permissions.

Conclusion

The project enhanced understanding of Terraform modules, AWS networking, and advanced Nginx configurations. The deployed infrastructure effectively supports load balancing, caching, and high availability.
